

PRD — Exercício de Programação em PHP

Projeto: Sistema de Usuários e Autenticação **Disciplina:** Design Patterns & Clean Code **Objetivo:**

Desenvolver um sistema simples em **PHP** aplicando **PSR-12**, **KISS** e **DRY**, para simular o cadastro, login e reset de senha de usuários.

Objetivo de Aprendizagem

- Praticar **Programação Orientada a Objetos (POO)** em PHP.
 - Aplicar **boas práticas de programação** (PSR-12, DRY, KISS).
 - Entender a importância de **validação de dados e segurança** (senhas com hash).
 - Desenvolver um sistema básico de autenticação com **fluxo realista**.
 - Documentar e versionar código em um **repositório GitHub**.
-

Escopo do Projeto

O sistema deve simular um **cadastro e autenticação de usuários**. Todos os dados serão fixos e representados em **arrays PHP**. Não é necessário banco de dados.

1. Lista de Usuários

- Deve conter:
 - **id** (inteiro)
 - **nome** (string)
 - **email** (string)
 - **senha** (hash)

Exemplo:

```
$usuarios = [  
    ['id' => 1, 'nome' => 'João Silva', 'email' => 'joao@email.com',  
     'senha' => 'SenhaForte1'],  
];
```

2. Funcionalidades

Cadastro de Usuário

- Validar se o e-mail é válido.
- Validar se a senha é forte (mín. 8 caracteres, ao menos 1 número e 1 maiúscula).
- Gerar hash seguro da senha com **password_hash**.
- Não permitir e-mails duplicados.

Login de Usuário

- Validar e-mail e senha fornecidos.
- Comparar senha com `password_verify`.
- Retornar mensagem de sucesso ou falha.

Reset de Senha

- Permitir atualizar a senha de um usuário existente.
- Aplicar novamente as regras de senha forte.
- Substituir pela nova senha com `password_hash`.

Casos de Uso

Caso 1 — Cadastro válido

- Entrada: nome `Maria Oliveira`, email `maria@email.com`, senha `Senha123`.
- Resultado esperado: usuário cadastrado com sucesso.

Caso 2 — Cadastro com e-mail inválido

- Entrada: nome `Pedro`, email `pedro@email`, senha `Senha123`.
- Resultado esperado: mensagem de erro → "E-mail inválido".

Caso 3 — Tentativa de login com senha errada

- Entrada: email `joao@email.com`, senha `Errada123`.
- Resultado esperado: mensagem de erro → "Credenciais inválidas".

Caso 4 — Reset de senha válido

- Entrada: id `1`, nova senha `NovaSenha1`.
- Resultado esperado: senha alterada com sucesso.

Caso 5 — Cadastro de usuário com e-mail duplicado

- Entrada: email já existente no array.
- Resultado esperado: mensagem de erro → "E-mail já está em uso".

Limitações

- Não precisa persistir dados (uso apenas de arrays).
- Quem topa um desafio pode usar SQLite
- Não precisa interface de login/usuário (pode ser via chamadas diretas em funções).
- Não precisa input via formulário → valores podem estar fixos em arrays/variáveis.
- Não usar frameworks externos. Apenas **PHP puro rodando no XAMPP**.

Entregáveis

1. Repositório no GitHub com:

- Código fonte em PHP seguindo **PSR-12**.
- Uso de **POO** (classes para **User**, **userManager**, **Validator**).
- Senhas com **password_hash** e **password_verify**.
- README.md contendo:
 - Nome e RA da dupla.
 - Instruções de execução (XAMPP).
 - Breve documentação (funcionalidades, regras de negócio, limitações).
- Pastas organizadas (**src/**, **docs/**).

2. README.md deve conter:

- Como rodar o projeto (**http://localhost/usuarios**).
- Lista de funcionalidades implementadas.
- Casos de uso (pelo menos 5 cenários de teste documentados).

Critérios de Avaliação

- Organização do código (PSR-12).
- Aplicação de **POO**, **DRY** e **KISS**.
- Segurança aplicada com **hash de senha**.
- Implementação de **pelo menos 5 cenários de teste**.
- Clareza e completude da documentação.
- Criatividade na solução.